

Analyzing decision algorithms and problems

Outline

Outline

1. Proof Methods

Outline

1. Proof Methods
2. Analysis of Algorithms

Outline

1. Proof Methods
2. Analysis of Algorithms
3. Analysis of Problems
 1. Decidability
 2. Tractability

Methods of Learning

(Disclaimer: I am not a trained epistemologist)

Methods of Learning

(Disclaimer: I am not a trained epistemologist)

Testimony

Methods of Learning

(Disclaimer: I am not a trained epistemologist)

Testimony

1. Read or hear a piece of knowledge
2. Decide whether to believe it (e.g. based on authority of speaker)
3. Learn by believing it

Methods of Learning

(Disclaimer: I am not a trained epistemologist)

Testimony

1. Read or hear a piece of knowledge
2. Decide whether to believe it (e.g. based on authority of speaker)
3. Learn by believing it

Perception (Scientific Method)

Methods of Learning

(Disclaimer: I am not a trained epistemologist)

Testimony

1. Read or hear a piece of knowledge
2. Decide whether to believe it (e.g. based on authority of speaker)
3. Learn by believing it

Perception (Scientific Method)

1. Formulate a question
2. Formulate a hypothesis
3. Test the hypothesis with an experiment
4. Learn by analyzing the results of the experiment

Methods of Learning

(Disclaimer: I am not a trained epistemologist)

Testimony

1. Read or hear a piece of knowledge
2. Decide whether to believe it (e.g. based on authority of speaker)
3. Learn by believing it

Perception (Scientific Method)

1. Formulate a question
2. Formulate a hypothesis
3. Test the hypothesis with an experiment
4. Learn by analyzing the results of the experiment

Reason (Mathematical Proof)

Methods of Learning

(Disclaimer: I am not a trained epistemologist)

Testimony

1. Read or hear a piece of knowledge
2. Decide whether to believe it (e.g. based on authority of speaker)
3. Learn by believing it

Perception (Scientific Method)

1. Formulate a question
2. Formulate a hypothesis
3. Test the hypothesis with an experiment
4. Learn by analyzing the results of the experiment

Reason (Mathematical Proof)

1. Formulate a question
2. Create a conjecture that answers the question
 1. *Define* all concepts
 2. State as a theorem
3. Prove or disprove the conjecture
4. Learn by considering the theorem/false conjecture and the proof

Methods of Learning

(Disclaimer: I am not a trained epistemologist)

Testimony

1. Read or hear a piece of knowledge
2. Decide whether to believe it (e.g. based on authority of speaker)
3. Learn by believing it

Perception (Scientific Method)

1. Formulate a question
2. Formulate a hypothesis
3. Test the hypothesis with an experiment
4. Learn by analyzing the results of the experiment

Reason (Mathematical Proof)

1. Formulate a question
2. Create a conjecture that answers the question
 1. *Define* all concepts
 2. State as a theorem
3. Prove or disprove the conjecture
4. Learn by considering the theorem/false conjecture and the proof

Most RL research fits into the scientific method.

Mathematical Proofs

A mathematical proof *must* (in Sunberg's words):

Mathematical Proofs

A mathematical proof *must* (in Sunberg's words):

Mathematical Proofs

A mathematical proof *must* (in Sunberg's words):

1. Be logically correct

Mathematical Proofs

A mathematical proof *must* (in Sunberg's words):

1. Be logically correct
2. Allow each step to be easily verified by a member of the intended audience

Mathematical Proofs

A mathematical proof *must* (in Sunberg's words):

1. Be logically correct
2. Allow each step to be easily verified by a member of the intended audience

"Nice-to-have"s for theorems, definitions, proofs:

1. General (Definitions and Theorems)
2. Easy for audience to understand
3. Follows patterns the audience is familiar with
4. Composable: allows for construction of other definitions, lemmas, theorems, proofs
5. Answers the question "why" (Proofs)
6. Interesting, engaging, fun for humans

Mathematical Proofs

A mathematical proof *must* (in Sunberg's words):

1. Be logically correct
2. Allow each step to be easily verified by a member of the intended audience

"Nice-to-have"s for theorems, definitions, proofs:

1. General (Definitions and Theorems)
2. Easy for audience to understand
3. Follows patterns the audience is familiar with
4. Composable: allows for construction of other definitions, lemmas, theorems, proofs
5. Answers the question "why" (Proofs)
6. Interesting, engaging, fun for humans

Proof Strategies

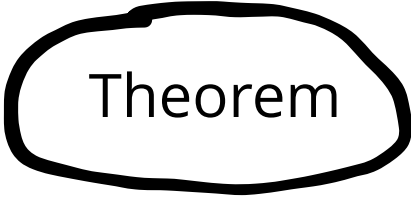
1. **Direct proof**
2. **Proof by induction**
3. **Proof by contradiction**
4. Proof by contrapositive
5. Proof by exhaustion

Direct Proof

Direct Proof

Definition

Direct Proof

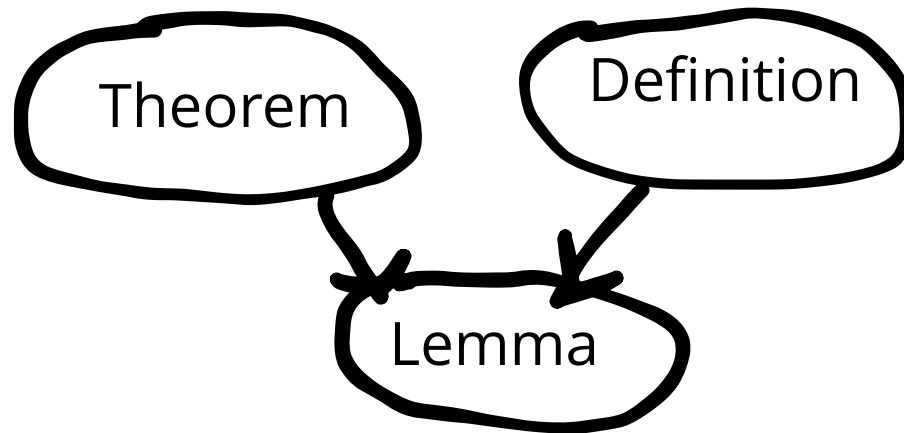


Theorem

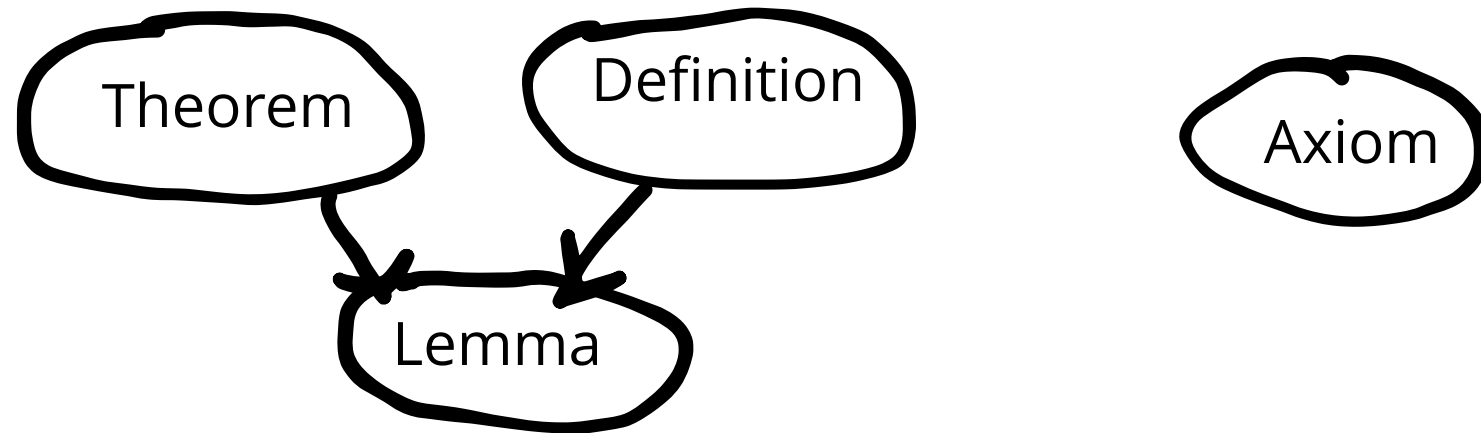


Definition

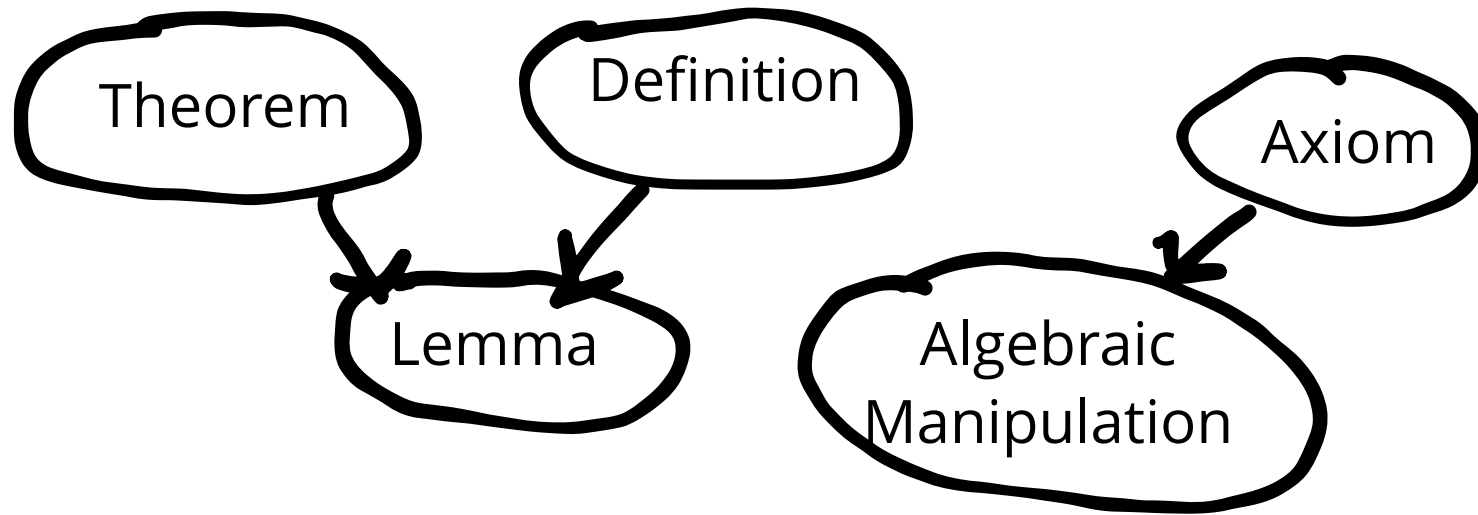
Direct Proof



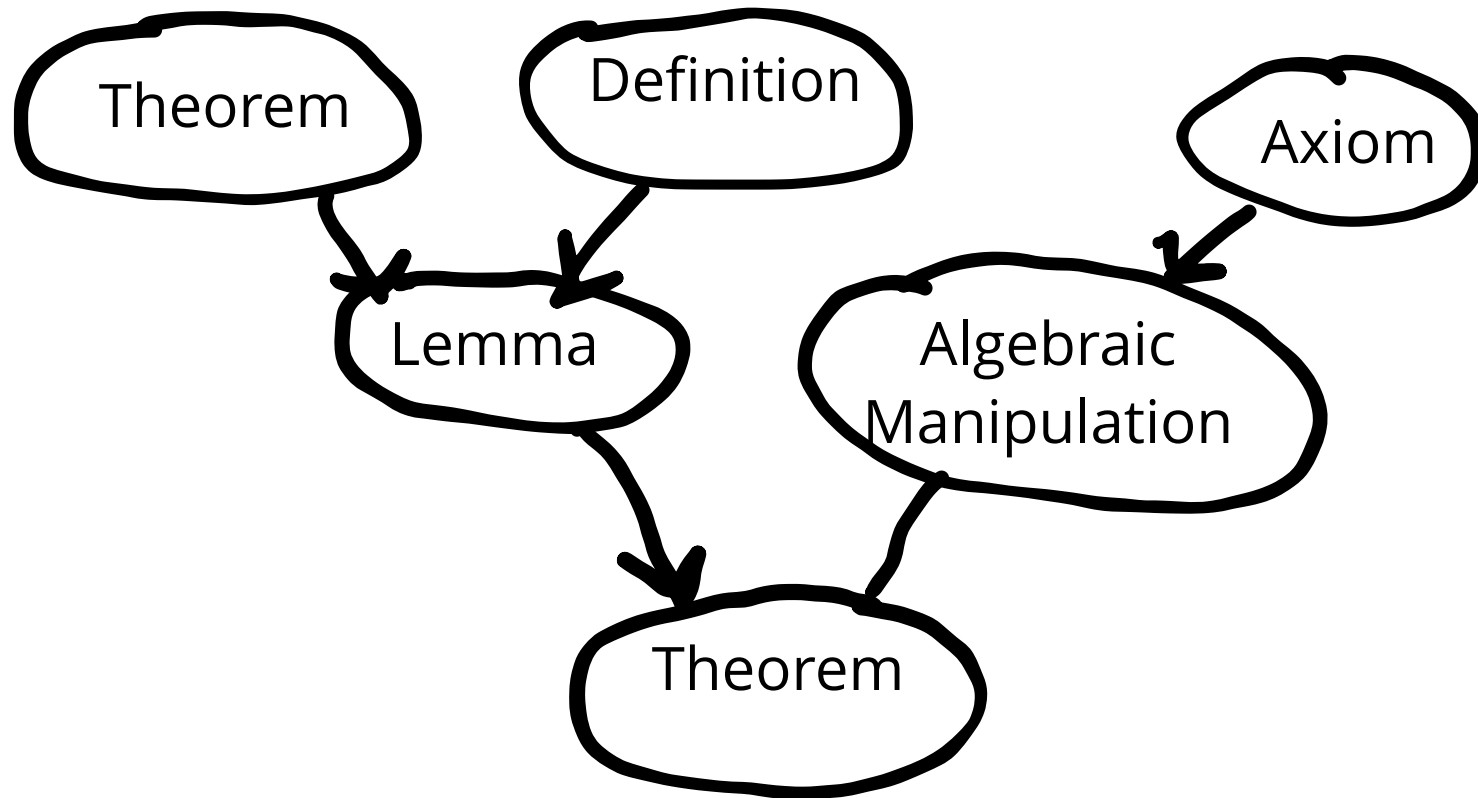
Direct Proof



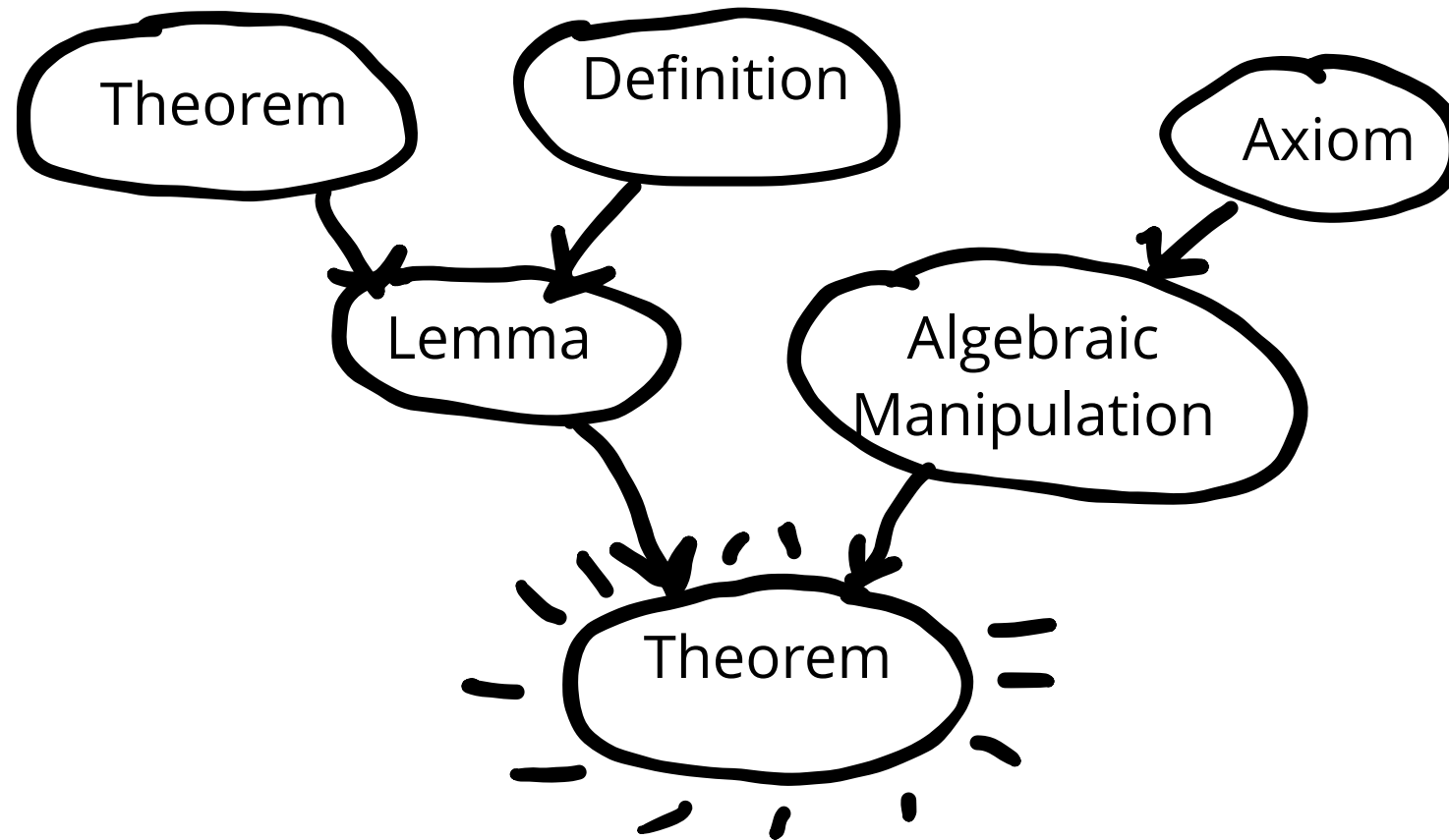
Direct Proof



Direct Proof



Direct Proof



Direct Proof Example

Direct Proof Example

Definition: An integer x is *even* if there exists an integer a such that $x = 2a$

Direct Proof Example

Definition: An integer x is *even* if there exists an integer a such that $x = 2a$

Theorem: The sum of two even integers is even.

Direct Proof Example

Definition: An integer x is *even* if there exists an integer a such that $x = 2a$

Theorem: The sum of two even integers is even.

Proof: Let x and y be even integers and z be their sum.
By definition, there exist integers a and b such that

$$x = 2a \text{ and}$$

$$y = 2b.$$

Then $z = x + y = 2a + 2b = 2(a + b)$. Since $a + b$ is an integer, z is even.

Proof by Induction

Proof by Induction

To prove $P(n)$ holds for every natural number n :

Proof by Induction

To prove $P(n)$ holds for every natural number n :

1. Prove $P(1)$ (the base case)

Proof by Induction

To prove $P(n)$ holds for every natural number n :

1. Prove $P(1)$ (the base case)
2. Prove that $P(k) \implies P(k + 1)$ (the induction step)

Proof by Induction Example

Proof by Induction Example

Theorem: For any natural number n , the sum of n and all natural numbers less than n is $\frac{n^2+n}{2}$

Proof by Induction Example

Theorem: For any natural number n , the sum of n and all natural numbers less than n is $\frac{n^2+n}{2}$

Proof: By induction:

Proof by Induction Example

Theorem: For any natural number n , the sum of n and all natural numbers less than n is $\frac{n^2+n}{2}$

Proof: By induction:

- Base case: The sum of 1 and all natural numbers less than 1 is $1 = \frac{1^2+1}{2}$

Proof by Induction Example

Theorem: For any natural number n , the sum of n and all natural numbers less than n is $\frac{n^2+n}{2}$

Proof: By induction:

- Base case: The sum of 1 and all natural numbers less than 1 is $1 = \frac{1^2+1}{2}$
- Induction step: If the sum of k and all natural numbers less than k is $\frac{k^2+k}{2}$, then the sum of $k+1$ and all natural numbers less than $k+1$ is $\frac{k^2+k}{2} + k + 1 = \frac{k^2+3k+2}{2} = \frac{(k+1)^2+k+1}{2}$

Proof by Contradiction

Proof by Contradiction

To prove proposition P :

Proof by Contradiction

To prove proposition P :

1. Assume P is false, i.e. assume $\neg P$

Proof by Contradiction

To prove proposition P :

1. Assume P is false, i.e. assume $\neg P$
2. Show that $\neg P$ implies two mutually contradictory assertions, Q and $\neg Q$

Proof by Contradiction

To prove proposition P :

1. Assume P is false, i.e. assume $\neg P$
2. Show that $\neg P$ implies two mutually contradictory assertions, Q and $\neg Q$
3. Q and $\neg Q$ cannot both be true (a "contradiction"), so P must be true

Proof by Contradiction

Example

Proof by Contradiction

Example

Theorem: $\sqrt{2}$ is irrational.

Proof by Contradiction

Example

Theorem: $\sqrt{2}$ is irrational.

Proof:

- Suppose $\sqrt{2}$ were rational. Then there exist positive integers a and b such that $\sqrt{2} = a/b$, **at least one of which is odd** (If a and b were both even, the fraction could be reduced).
- Since $a^2 = 2b^2$, a^2 must be even, and thus a is even (since the square of an odd number is odd), and there exists some k such that $a = 2k$.
- Then $2b^2 = (2k)^2 = 4k^2$, so $b^2 = 2k^2$.
- Therefore b^2 is even, and b is also even. **Thus, both a and b are even.** This is a contradiction.

Algorithm Analysis

Computational Complexity: What is the number of basic operations an algorithm will use?

Asymptotic Notation

Asymptotic Notation

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\} .$

Asymptotic Notation

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\} .$

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\} .$

Asymptotic Notation

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\} .$

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\} .$

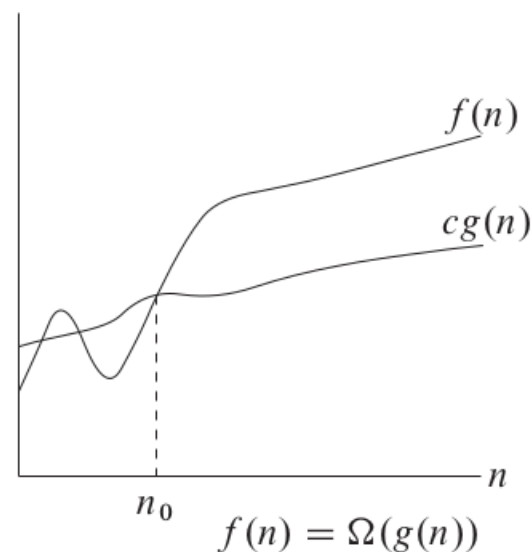
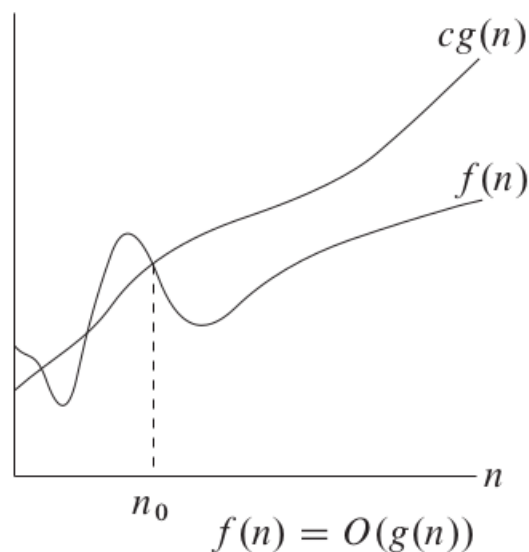
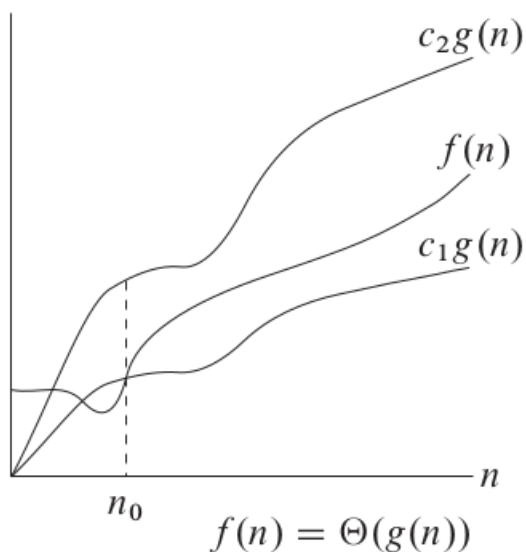
$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that}$
 $0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\} .^1$

Asymptotic Notation

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\} .$

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$
 $0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\} .$

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that}$
 $0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\} .^1$



Exercise: Complexity of Value Iteration for fixed horizon

Fixed Horizon Case:

for $t \in \{H - 1, \dots, 0\}$

for $s \in \mathcal{S}$

$$V_t^*(s) = \max_{a \in \mathcal{A}} (R(s, a) + \gamma \sum_{s' \in \mathcal{S}} T(s' \mid s, a) V_{t+1}^*(s'))$$

end

end

Exercise: Complexity of Value Iteration for fixed horizon

Fixed Horizon Case:

for $t \in \{H - 1, \dots, 0\}$

H

for $s \in \mathcal{S}$

$$V_t^*(s) = \max_{a \in \mathcal{A}} (R(s, a) + \gamma \sum_{s' \in \mathcal{S}} T(s' \mid s, a) V_{t+1}^*(s'))$$

end

end

Exercise: Complexity of Value Iteration for fixed horizon

Fixed Horizon Case:

for $t \in \{H - 1, \dots, 0\}$

H

for $s \in \mathcal{S}$

$|\mathcal{S}|$

$$V_t^*(s) = \max_{a \in \mathcal{A}} (R(s, a) + \gamma \sum_{s' \in \mathcal{S}} T(s' \mid s, a) V_{t+1}^*(s'))$$

end

end

Exercise: Complexity of Value Iteration for fixed horizon

Fixed Horizon Case:

for $t \in \{H - 1, \dots, 0\}$

H

for $s \in \mathcal{S}$

$|\mathcal{S}|$

$$V_t^*(s) = \max_{a \in \mathcal{A}} (R(s, a) + \gamma \sum_{s' \in \mathcal{S}} T(s' \mid s, a) V_{t+1}^*(s'))$$

$O(|\mathcal{A}||\mathcal{S}|)$

end

end

Exercise: Complexity of Value Iteration for fixed horizon

Fixed Horizon Case:

for $t \in \{H - 1, \dots, 0\}$

H

for $s \in \mathcal{S}$

$|\mathcal{S}|$

$$V_t^*(s) = \max_{a \in \mathcal{A}} (R(s, a) + \gamma \sum_{s' \in \mathcal{S}} T(s' \mid s, a) V_{t+1}^*(s'))$$

$O(|\mathcal{A}||\mathcal{S}|)$

end

end

$O(H|\mathcal{A}||\mathcal{S}|^2)$

Problem Analysis

Which problems can be solved and how hard are they?

Part 1: Decidability

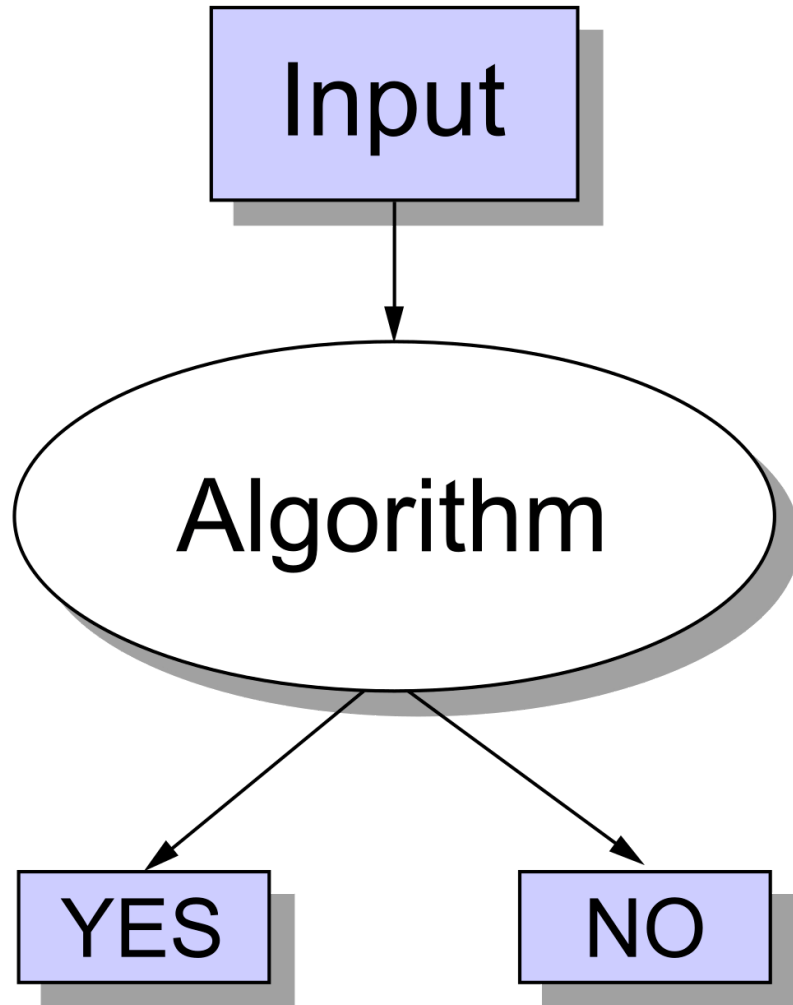
Decision Problems

Decision Problems

- A *Decision Problem* is a problem that has a yes-no answer for a set of inputs

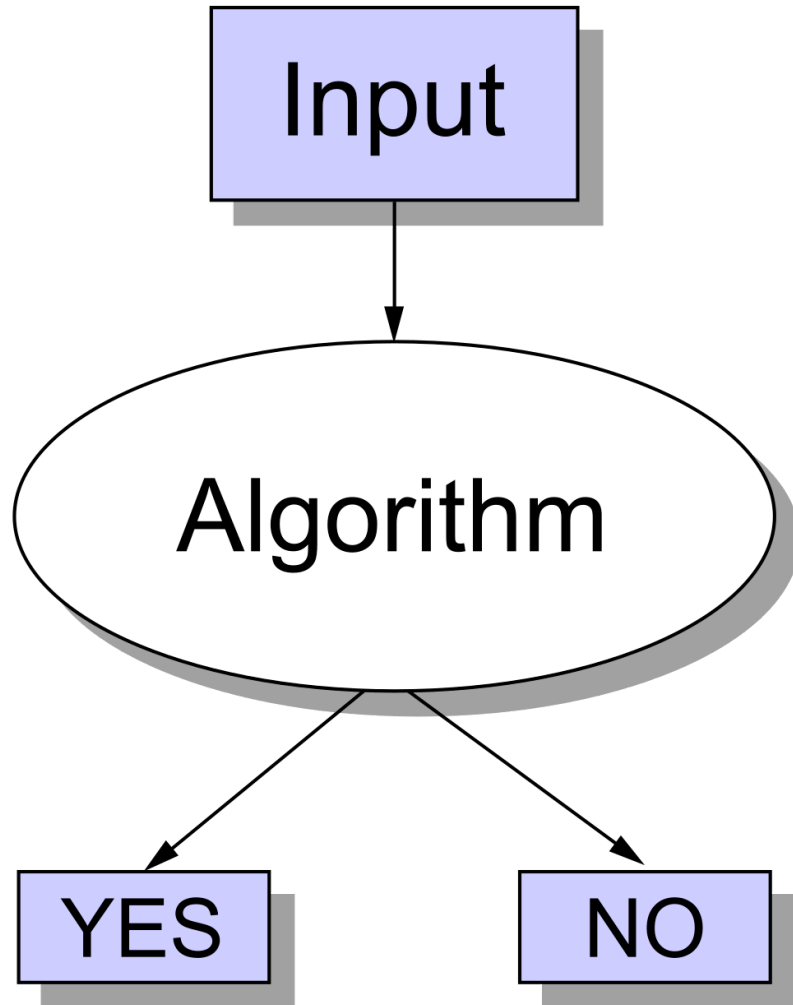
Decision Problems

- A *Decision Problem* is a problem that has a yes-no answer for a set of inputs

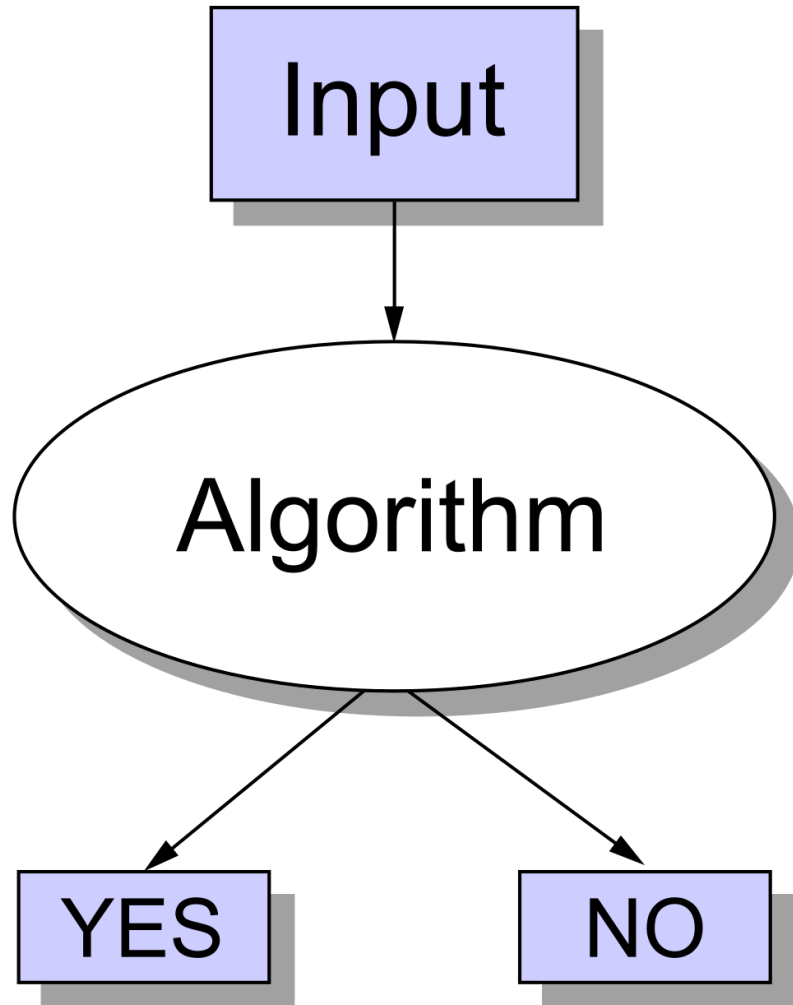


Decision Problems

- A *Decision Problem* is a problem that has a yes-no answer for a set of inputs
- Most of the problems that we study in DMU are optimization problems

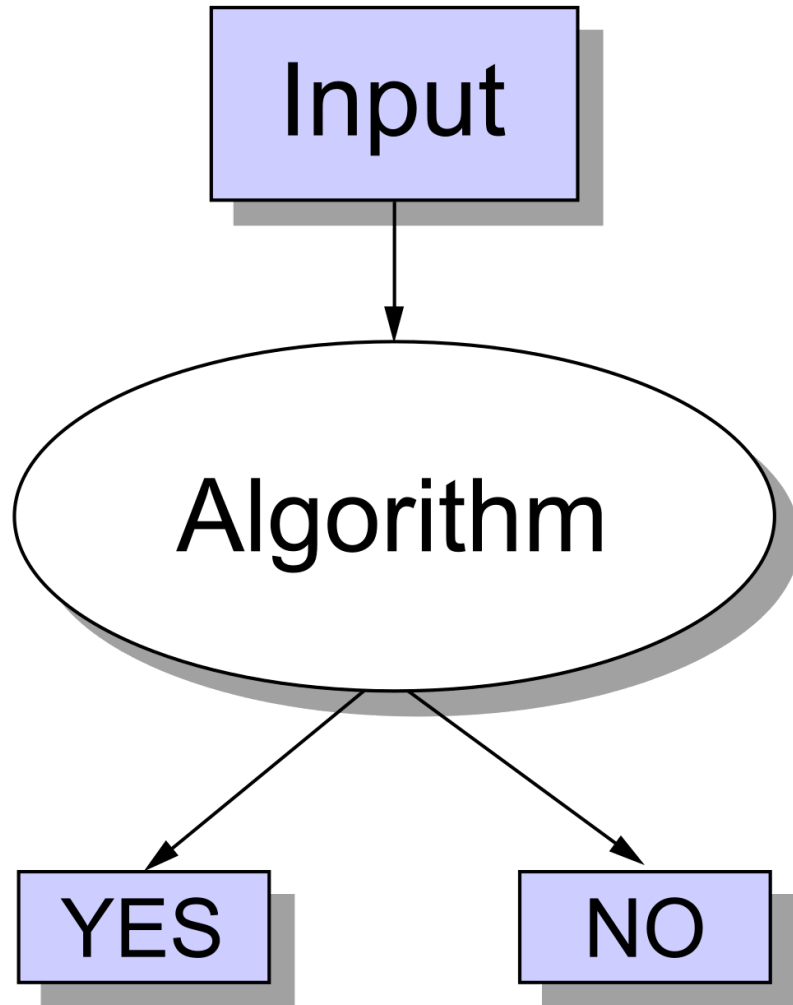


Decision Problems



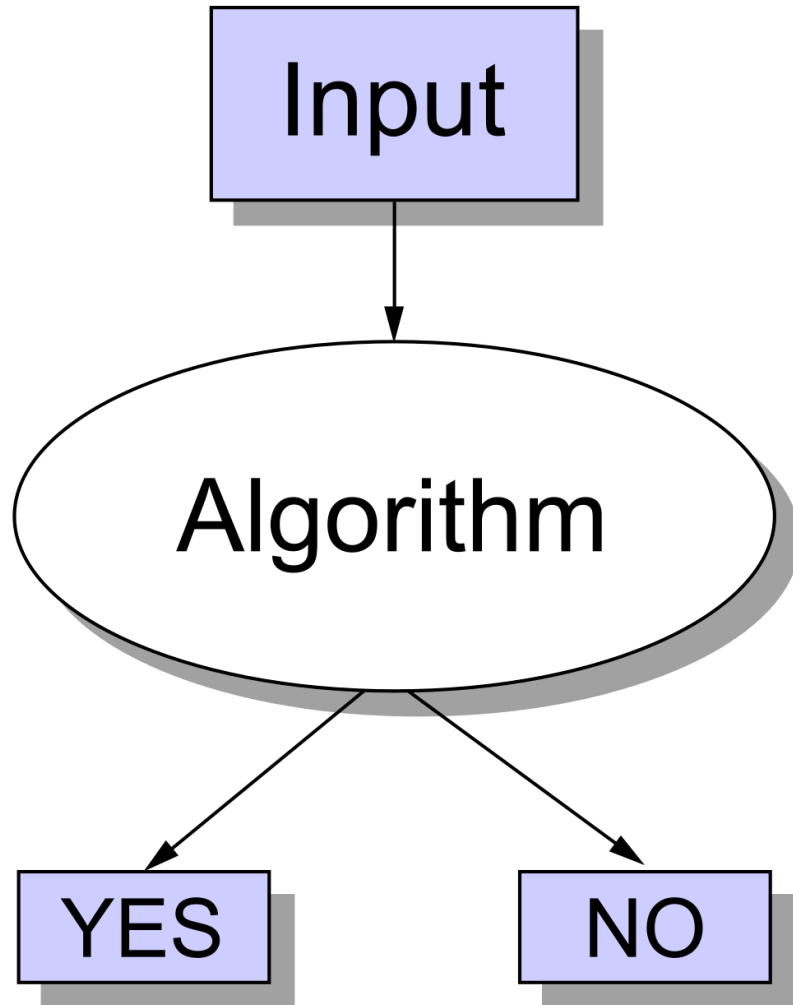
- A *Decision Problem* is a problem that has a yes-no answer for a set of inputs
- Most of the problems that we study in DMU are optimization problems
- An optimization problem can be cast as a decision problem by asking if there is a solution that achieves a given value.

Decision Problems



- A *Decision Problem* is a problem that has a yes-no answer for a set of inputs
- Most of the problems that we study in DMU are optimization problems
- An optimization problem can be cast as a decision problem by asking if there is a solution that achieves a given value.
- An optimization algorithm can answer this question by optimizing and checking whether the optimal value is above the given value.

Decision Problems



- A *Decision Problem* is a problem that has a yes-no answer for a set of inputs
- Most of the problems that we study in DMU are optimization problems
- An optimization problem can be cast as a decision problem by asking if there is a solution that achieves a given value.
- An optimization algorithm can answer this question by optimizing and checking whether the optimal value is above the given value.

Finite-Horizon MDP Decision Problem:

- Inputs: $S, A, R, T, \gamma, b_0, H, \underline{V}$
- Output: Is $E_{s \sim b_0} V_0^*(s) \geq \underline{V}$?

Decidability

Decidability

A problem is *decidable* if there exists an algorithm that will correctly decide every instance of a problem in finite time.

Decidability

A problem is *decidable* if there exists an algorithm that will correctly decide every instance of a problem in finite time.

Is the finite horizon MDP decision problem decidable?

Decidability

Decidability

Are there problems that are undecidable?

Decidability

Are there problems that are undecidable?

Yes! Example: Halting Problem:

- Inputs: program f
- Output: will f finish in finite time?

Decidability

Are there problems that are undecidable?

Yes! Example: Halting Problem:

- Inputs: program f
- Output: will f finish in finite time?

How can we prove that the halting problem is undecidable?

Decidability

Are there problems that are undecidable?

Yes! Example: Halting Problem:

- Inputs: program f
- Output: will f finish in finite time?

How can we prove that the halting problem is undecidable?

Basic idea (this is NOT a rigorous proof): Proof by contradiction

- Assume $h(f)$ returns true iff f halts
- Define a function g that loops forever if $h(g)$ is true (doing this rigorously is more technical than we will get into)
- Then $h(g)$ cannot return true or false, a contradiction

Proof by Reduction

Proof by Reduction

We know the halting problem is undecidable. Can we use this to prove that other problems are undecidable?

Proof by Reduction

We know the halting problem is undecidable. Can we use this to prove that other problems are undecidable?

Yes! via a Reduction.

Problem A is reducible to problem B , denoted $A \leq B$, if every instance of A can be translated into an instance of B in finite time.

Proof by Reduction

We know the halting problem is undecidable. Can we use this to prove that other problems are undecidable?

Yes! via a Reduction.

Problem A is reducible to problem B , denoted $A \leq B$, if every instance of A can be translated into an instance of B in finite time.

If the halting problem is reducible to B , then B is undecidable.

Part 2: Tractability

Tractability

Tractability

$$O(1)$$

$$O(\log(n))$$

$$O(n)$$

$$O(n^2)$$

$$O(n^k)$$

$$O(k^n)$$

Tractability

What definition should we use to separate tractable from intractable problems?

$$O(1)$$

$$O(\log(n))$$

$$O(n)$$

$$O(n^2)$$

$$O(n^k)$$

$$O(k^n)$$

Tractability

What definition should we use to separate tractable from intractable problems?

$$O(1)$$

$$O(\log(n))$$

$$O(n)$$

$$O(n^2)$$

$$O(n^k)$$

$$O(k^n)$$

Table 2.1 The running times (rounded up) of different algorithms on inputs of increasing size, for a processor performing a million high-level instructions per second. In cases where the running time exceeds 10^{25} years, we simply record the algorithm as taking a very long time.

	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	4 sec
$n = 30$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	18 min	10^{25} years
$n = 50$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	11 min	36 years	very long
$n = 100$	< 1 sec	< 1 sec	< 1 sec	1 sec	12,892 years	10^{17} years	very long
$n = 1,000$	< 1 sec	< 1 sec	1 sec	18 min	very long	very long	very long
$n = 10,000$	< 1 sec	< 1 sec	2 min	12 days	very long	very long	very long
$n = 100,000$	< 1 sec	2 sec	3 hours	32 years	very long	very long	very long
$n = 1,000,000$	1 sec	20 sec	12 days	31,710 years	very long	very long	very long

Tractability

What definition should we use to separate tractable from intractable problems?

$$O(1)$$

$$O(\log(n))$$

$$O(n)$$

$$O(n^2)$$

$$O(n^k)$$

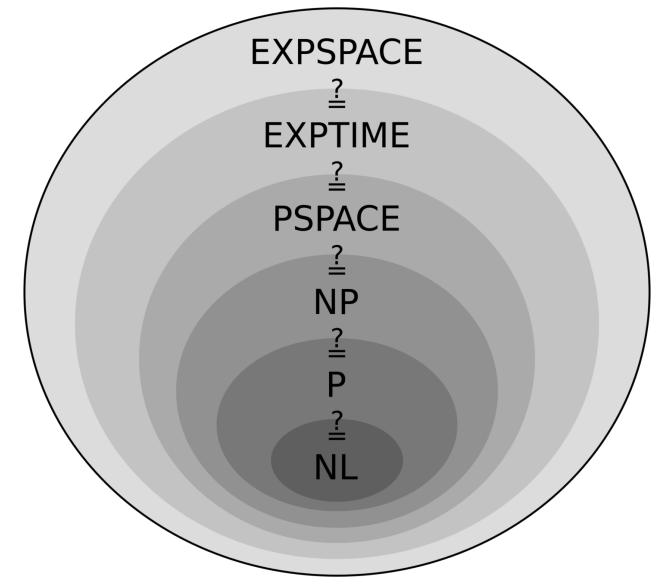
$$O(k^n)$$

Table 2.1 The running times (rounded up) of different algorithms on inputs of increasing size, for a processor performing a million high-level instructions per second. In cases where the running time exceeds 10^{25} years, we simply record the algorithm as taking a very long time.

	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	4 sec
$n = 30$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	18 min	10^{25} years
$n = 50$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	11 min	36 years	very long
$n = 100$	< 1 sec	< 1 sec	< 1 sec	1 sec	12,892 years	10^{17} years	very long
$n = 1,000$	< 1 sec	< 1 sec	1 sec	18 min	very long	very long	very long
$n = 10,000$	< 1 sec	< 1 sec	2 min	12 days	very long	very long	very long
$n = 100,000$	< 1 sec	2 sec	3 hours	32 years	very long	very long	very long
$n = 1,000,000$	1 sec	20 sec	12 days	31,710 years	very long	very long	very long

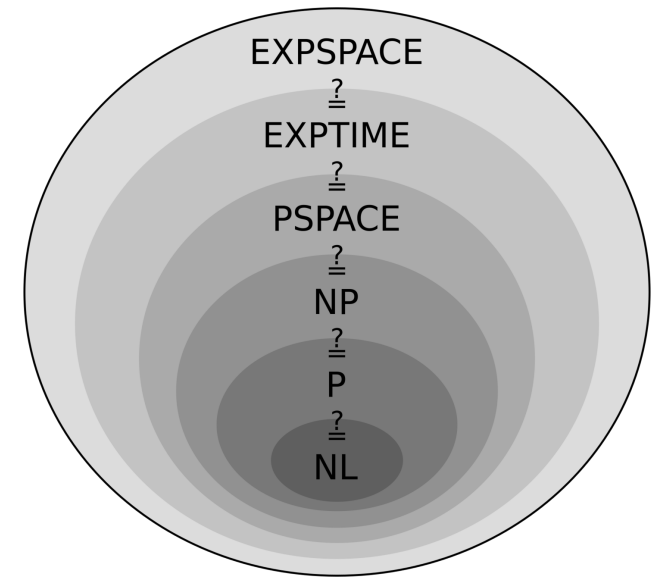
We say that a problem is *tractable* if it can be solved in $O(n^k)$ for some k .

Complexity Classes



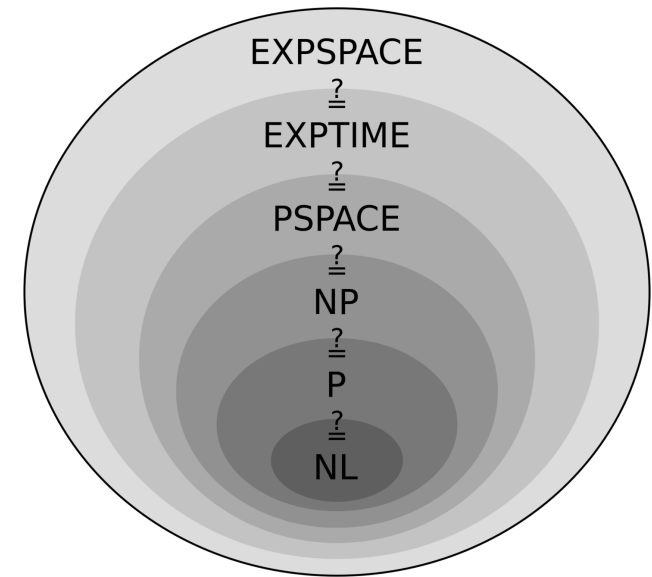
Complexity Classes

- P: Any instance can be solved in polynomial time in the size of the input
 - Is the Finite Horizon MDP a member of P?



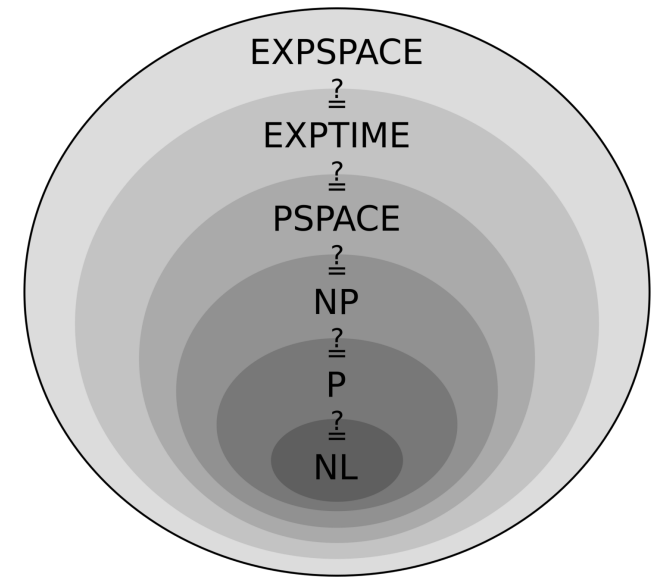
Complexity Classes

- P: Any instance can be solved in polynomial time in the size of the input
 - Is the Finite Horizon MDP a member of P?
- EXPTIME: Any instance can be solved in exponential time in the size of the input



Complexity Classes

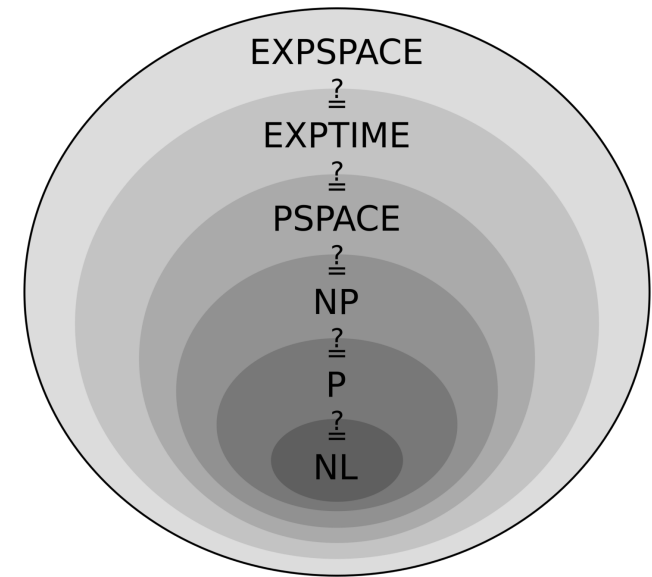
- P: Any instance can be solved in polynomial time in the size of the input
 - Is the Finite Horizon MDP a member of P?
- EXPTIME: Any instance can be solved in exponential time in the size of the input



How can we prove that a problem is a member of a class?

Complexity Classes

- P: Any instance can be solved in polynomial time in the size of the input
 - Is the Finite Horizon MDP a member of P?
- EXPTIME: Any instance can be solved in exponential time in the size of the input



How can we prove that a problem is a member of a class?

By finding an algorithm that solves the problem within the computational limit.

The NP Complexity Class

The NP Complexity Class

- "Nondeterministic Polynomial"

The NP Complexity Class

- "Nondeterministic Polynomial"
- Solution can be *verified* in polynomial time

The NP Complexity Class

- "Nondeterministic Polynomial"
- Solution can be *verified* in polynomial time
- Is $P \subseteq NP$?

The NP Complexity Class

- "Nondeterministic Polynomial"
- Solution can be *verified* in polynomial time
- Is $P \subseteq NP$?
- Example of a problem that is in NP but not P?

The NP Complexity Class

- "Nondeterministic Polynomial"
- Solution can be *verified* in polynomial time
- Is $P \subseteq NP$?
- Example of a problem that is in NP but not P?

$$P \subseteq NP$$

The NP Complexity Class

- "Nondeterministic Polynomial"
- Solution can be *verified* in polynomial time
- Is $P \subseteq NP$?
- Example of a problem that is in NP but not P?

$$P \subset NP$$

(We think - this is one of the biggest
unsolved problems in CS)

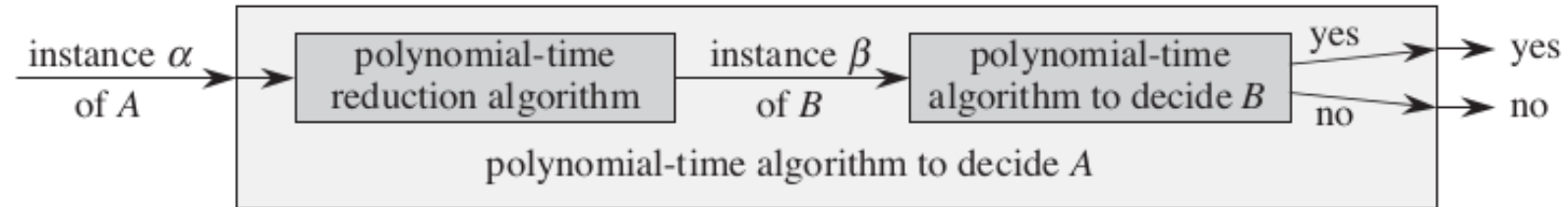
Polynomial Reduction

Polynomial Reduction

How can we prove that problem B is at least as hard as problem A?

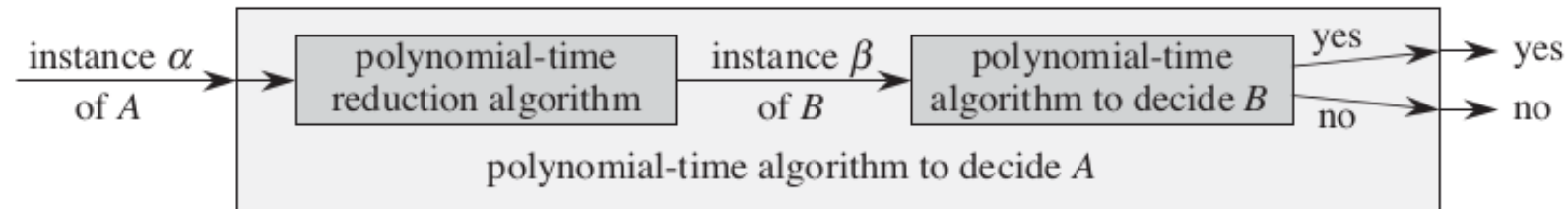
Polynomial Reduction

How can we prove that problem B is at least as hard as problem A?



Polynomial Reduction

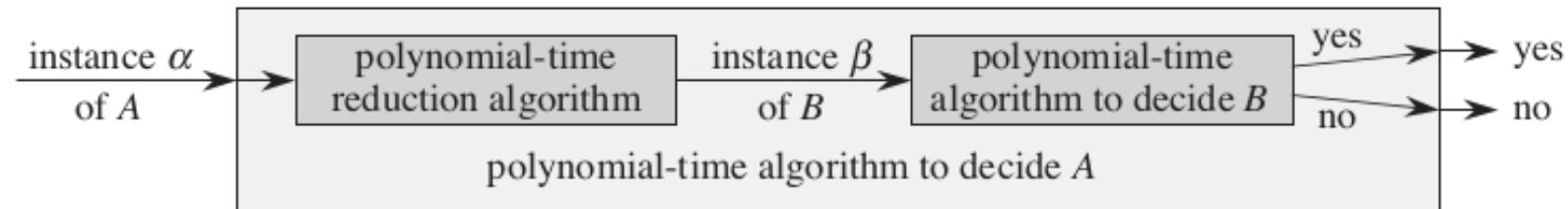
How can we prove that problem B is at least as hard as problem A?



$$A \leq_P B$$

Polynomial Reduction

How can we prove that problem B is at least as hard as problem A?



$$A \leq_P B$$

A is no harder than B

NP Completeness

Example proving that a problem is NP-complete is in section 34.3 of "Introduction to Algorithms"

NP Completeness

Problem A is said to be *NP-complete* if $A \in \text{NP}$ and
$$B \leq_P A \quad \forall B \in \text{NP}.$$

NP Completeness

Problem A is said to be *NP-complete* if $A \in \text{NP}$ and

$$B \leq_P A \quad \forall B \in \text{NP}.$$

Example: 3-SAT:

Given a set of clauses, $C_1, \dots, C_k$, each of length 3, over a set of binary variables $X = \{x_1, \dots, x_n\}$, does there exist a satisfying truth assignment?

NP Completeness

Problem A is said to be *NP-complete* if $A \in \text{NP}$ and

$$B \leq_P A \quad \forall B \in \text{NP}.$$

Example: 3-SAT:

Given a set of clauses, $C_1, \dots, C_k$, each of length 3, over a set of binary variables $X = \{x_1, \dots, x_n\}$, does there exist a satisfying truth assignment?

$$(x_1 \vee x_3 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$$

NP Completeness

Problem A is said to be *NP-complete* if $A \in \text{NP}$ and

$$B \leq_P A \quad \forall B \in \text{NP}.$$

Example: 3-SAT:

Given a set of clauses, $C_1, \dots, C_k$, each of length 3, over a set of binary variables $X = \{x_1, \dots, x_n\}$, does there exist a satisfying truth assignment?

$$(x_1 \vee x_3 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$$

- 3-SAT is NP-complete (Cook, 1971)

NP Completeness

Problem A is said to be *NP-complete* if $A \in \text{NP}$ and

$$B \leq_P A \quad \forall B \in \text{NP}.$$

Example: 3-SAT:

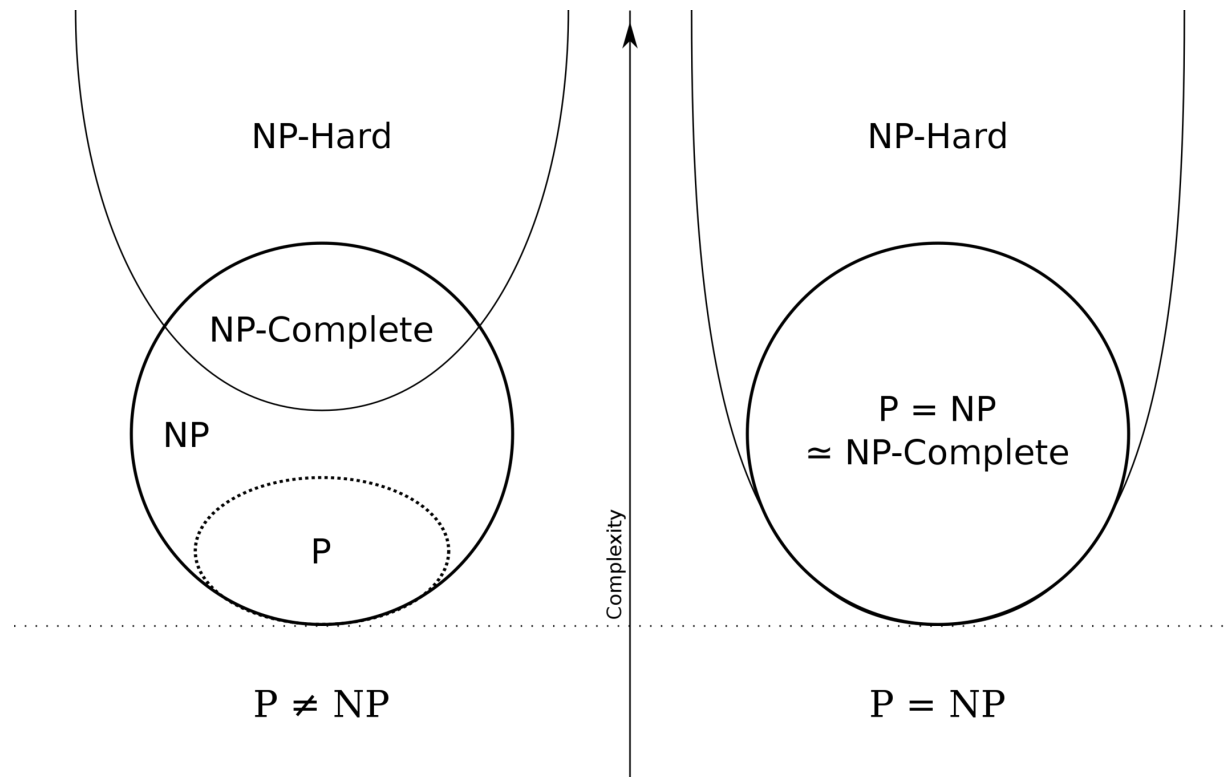
Given a set of clauses, $C_1, \dots, C_k$, each of length 3, over a set of binary variables $X = \{x_1, \dots, x_n\}$, does there exist a satisfying truth assignment?

$$(x_1 \vee x_3 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$$

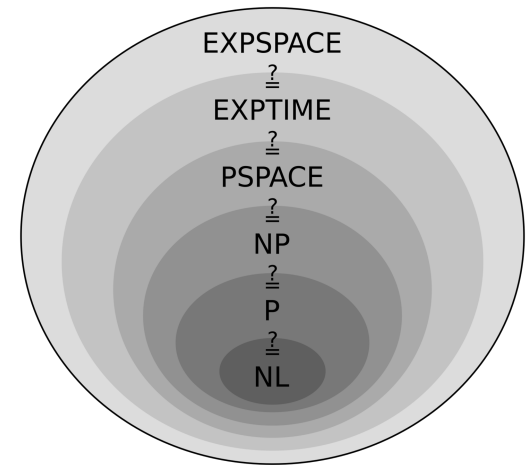
- 3-SAT is NP-complete (Cook, 1971)
- If 3-SAT can be reduced to $A \in \text{NP}$, A is also NP-complete!

NP-Hardness

- NP-hard: at least as hard as all of the problems in NP
- NP-complete: NP-hard and in NP

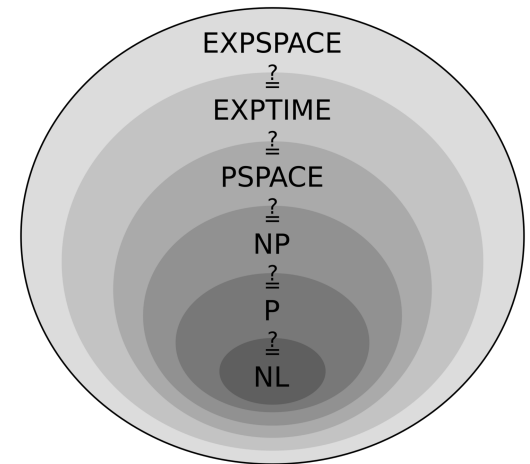


PSPACE Complexity Class



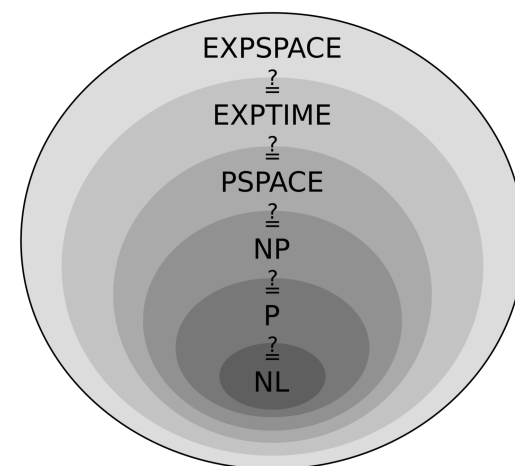
PSPACE Complexity Class

- PSPACE: Set of all problems that can be solved using polynomial *space*



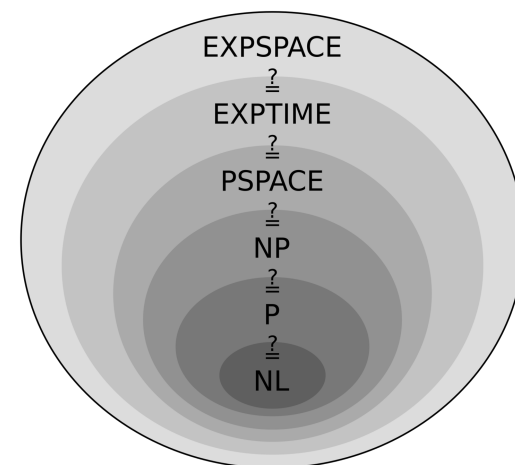
PSPACE Complexity Class

- PSPACE: Set of all problems that can be solved using polynomial *space*
- Observation 1: $P \subseteq PSPACE$



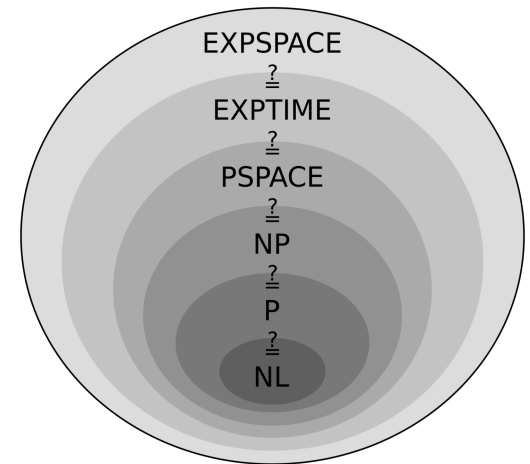
PSPACE Complexity Class

- PSPACE: Set of all problems that can be solved using polynomial *space*
- Observation 1: $P \subseteq PSPACE$
- Observation 2: There are algorithms that use exponential time, but only polynomial space



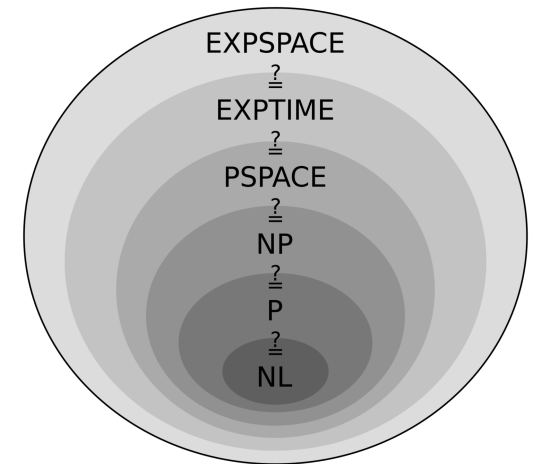
PSPACE Complexity Class

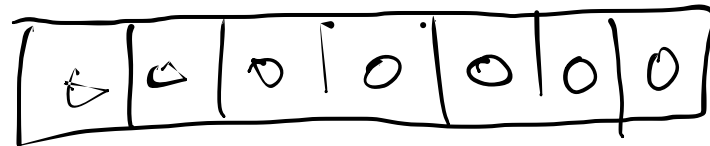
- PSPACE: Set of all problems that can be solved using polynomial *space*
- Observation 1: $P \subseteq PSPACE$
- Observation 2: There are algorithms that use exponential time, but only polynomial space
- Observation 3: There is an algorithm that solves 3-SAT using polynomial space, therefore $NP \subseteq PSPACE$



PSPACE Complexity Class

- PSPACE: Set of all problems that can be solved using polynomial *space*
- Observation 1: $P \subseteq PSPACE$
- Observation 2: There are algorithms that use exponential time, but only polynomial space
- Observation 3: There is an algorithm that solves 3-SAT using polynomial space, therefore $NP \subseteq PSPACE$
- $NP \subset PSPACE$???





0 0 0 0 0 0 1

0 0 0 0 0 1 0

PSPACE-Complete Problems

PSPACE-Complete Problems

QSAT: Let $\Phi(x_1, \dots, x_n)$ be a 3-SAT expression with odd n , then

$$\exists x_1 \forall x_2 \cdots \exists x_{n-2} \forall x_n \Phi(x_1, \dots, x_n)?$$

PSPACE-Complete Problems

QSAT: Let $\Phi(x_1, \dots, x_n)$ be a 3-SAT expression with odd n , then

$$\exists x_1 \forall x_2 \cdots \exists x_{n-2} \forall x_n \Phi(x_1, \dots, x_n)?$$

(for comparison 3-SAT asked $\exists x_1 \exists x_2 \cdots \exists x_{n-2} \exists x_n \Phi(x_1, \dots, x_n)?$)

PSPACE-Complete Problems

QSAT: Let $\Phi(x_1, \dots, x_n)$ be a 3-SAT expression with odd n , then

$$\exists x_1 \forall x_2 \cdots \exists x_{n-2} \forall x_n \Phi(x_1, \dots, x_n)?$$

(for comparison 3-SAT asked $\exists x_1 \exists x_2 \cdots \exists x_{n-2} \exists x_n \Phi(x_1, \dots, x_n)$?)

- QSAT is PSPACE-complete (Stockmeyer and Meyer, 1973)

PSPACE-Complete Problems

QSAT: Let $\Phi(x_1, \dots, x_n)$ be a 3-SAT expression with odd n , then

$$\exists x_1 \forall x_2 \cdots \exists x_{n-2} \forall x_n \Phi(x_1, \dots, x_n)?$$

(for comparison 3-SAT asked $\exists x_1 \exists x_2 \cdots \exists x_{n-2} \exists x_n \Phi(x_1, \dots, x_n)$?)

- QSAT is PSPACE-complete (Stockmeyer and Meyer, 1973)
- If QSAT can be reduced to A, A is PSPACE-hard

Review

Review

1. Three most common proof methods:

1. Direct
2. Induction
3. Contradiction

Review

1. Three most common proof methods:

1. Direct
2. Induction
3. Contradiction

2. Algorithm Complexity

Review

1. Three most common proof methods:

1. Direct
2. Induction
3. Contradiction

2. Algorithm Complexity

3. Decision Problems: Yes or no answer, can reframe optimization as this

Review

1. Three most common proof methods:

1. Direct
2. Induction
3. Contradiction

2. Algorithm Complexity

3. Decision Problems: Yes or no answer, can reframe optimization as this

4. Decidability

1. Problem is decidable if there is an finite-time algorithm that will decide
2. There are undecidable problems like the halting problem
3. Can prove problems are decidable by providing an algorithm, undecidable by reducing an undecidable problem to it.

Review

1. Three most common proof methods:

1. Direct
2. Induction
3. Contradiction

2. Algorithm Complexity

3. Decision Problems: Yes or no answer, can reframe optimization as this

4. Decidability

1. Problem is decidable if there is an finite-time algorithm that will decide
2. There are undecidable problems like the halting problem
3. Can prove problems are decidable by providing an algorithm, undecidable by reducing an undecidable problem to it.

5. Tractability

1. Problem is tractable if there is a polynomial-time algorithm that decides it
2. Complexity Classes: P, NP, PSPACE, etc.
3. Can prove problems are in a class by providing an algorithm, complete by reducing a complete problem to it and proving it is in the class